InMarket AI Builder — Tech Interview Prep

> **Format:** Tech + behavioral, ~60 min. Focus: AI tools experience, project walkthroughs, execution & systems thinking, collaboration.
> **Your posture:** Confident entry-to-mid practitioner who built real production AI systems, moves fast, and knows when AI helps vs. hurts.

Table of Contents

1. Tell Me About Yourself / Role Fit
2. AI Tools & AI-Native IDEs
3. The Take-Home Project (Consumer Trend Intelligence Agent)
4. LLM & Agent Architecture
5. Systems Thinking & Execution
6. Past Projects & Real-World Experience
7. Deployment, DevOps & Cloud
8. Trade-offs & Decision Making
9. Communication with Non-Technical Stakeholders
10. Behavioral / Situational

---

1. Tell Me About Yourself / Role Fit

Q: Walk me through your background and why you're a fit for this role.

I'm a full-stack software engineer with about four years of experience, and the last two have been almost entirely in AI-augmented systems — building LLM pipelines, agent backends, internal tools, and automation workflows.

At my most recent job at Epik Solutions, I was embedded in an enterprise security team and built AI-driven triage pipelines on top of Azure OpenAI that processed over a thousand security events a day and cut analyst workload by about 30%. Before that, at Samay Systems, I was a founding engineer on a compliance SaaS platform where I built the entire LLM layer — LangChain, MCP, AWS Bedrock — to automate evidence collection and scoring, which took manual auditor effort down by 70%. The infrastructure for all of that was AWS CDK in TypeScript — Node.js ecosystem — 15+ stacks covering VPC, Lambda, ECS Fargate, and AppRunner.

The baseline engineering practices have been consistent throughout: RESTful API design for every service boundary, Git with PR-gated CI/CD (GitHub Actions runs tests on every PR, image builds on merge), and Agile sprint cycles with iterative feedback from real users. The AI layer is what I go

deepest on, but I build it the same way I'd build any production system.

The reason I'm excited about this role specifically is that InMarket is building internal tools powered by AI, which is exactly the kind of work I find most useful. Not AI for demo purposes — AI that actually removes friction for real teams. That's been the through-line in everything I've built.

Q: The JD says "entry level" but also lists real experience requirements. How do you square that?

Honestly, I think the entry-level label is about organizational level and pay band — it's a contract role, not a senior staff hire. The technical bar is real, and I think my background clears it. I've built multi-tenant SaaS systems from zero, I've integrated LLMs into production workflows, and I've shipped tools that actual users depended on. The fact that they want someone who can move fast, pick things up quickly, and build without a lot of hand-holding — that's exactly how I operate.

---

2. AI Tools & AI-Native IDEs

Q: What AI tools and IDEs do you actually use day-to-day? Walk me through your setup.

My primary environment is **GitHub Copilot Chat** inside VS Code. I use it at almost every step — asking it to scaffold boilerplate, explain an unfamiliar library, review logic I've written, or generate test cases. The key for me is that I treat it as a pair programmer, not a code generator. I still drive the architecture; Copilot fills in the tedious parts.

I've also used **Cursor** for projects where I wanted a more agentic flow — the "composer" feature where you describe a module and it generates the file structure is great for getting something buildable in the first 30 minutes. I used **Augment Code** at Samay Systems, especially for navigating a large multi-repo codebase — it indexes the whole project context so when you're working in a Lambda function you can ask it about how a CDK stack three files away configures that function's environment. And I've used **Claude** directly for things like reviewing a whole architecture document or working through a complex prompt engineering problem where I need long-context reasoning.

For the take-home project specifically, I used GitHub Copilot the whole way through. I'd describe the component — for example, "FastAPI endpoint that takes a query and returns a structured Pydantic model representing YouTube video metadata" — and iterate on what it returned. The agentic IDE is best when your prompts are specific and grounded, not vague.

Q: What's the most effective way you've used an AI IDE to speed up development? Give a concrete example.

The clearest example is the MCP server I built for the take-home. I needed eight separate tool endpoints, each hitting a different external API with its own auth, pagination, and error handling.

Doing that from scratch would have taken me a full day.

What I did instead: I wrote one complete endpoint by hand so I had the pattern exactly right — Pydantic input model, service function, httpx call, structured response model. Then I prompted Copilot with: "Here's the pattern from `search_youtube_discussions`. Create `get_market_demographics` using the US Census ACS API following the same structure, with these input fields and this response shape." It generated ~80% of the code correctly the first time. I reviewed, fixed the API-specific quirks, and moved on. I repeated that for all eight tools in about two hours.

The insight is: **pattern-first, then delegate**. Once you have one well-built example, the AI can replicate the pattern reliably. If you ask it to figure out the pattern itself, you get inconsistent results.

Q: Where do AI tools NOT help, or even hurt you?

A few places:

System design decisions. Copilot will happily generate a monolith if you don't push back. It doesn't know your scalability requirements, your team's constraints, or your deployment environment. I always design the architecture first on paper before touching the IDE.

Debugging subtle runtime issues. When a Go runbook at Epik was silently swallowing errors and returning success codes, Copilot's suggestions were plausible but wrong — it was pattern-matching to similar-looking code rather than understanding the actual execution context. I had to trace through the runtime manually, understand the goroutine lifecycle, and then fix it myself. AI caught me a red herring there before I found the real issue.

Security-sensitive code. I don't let the AI write authentication logic, secret handling, or anything touching PII without heavy review. It tends to generate functional but insecure patterns — hardcoded credentials in examples, missing input validation, that kind of thing.

Q: The JD specifically mentions Cursor, Windsurf, and Claude Code. Have you used them?

Yes. I used Cursor on the take-home project's prototype phase — the challenge folder in my submission shows the rough first pass I built using Cursor's composer feature to scaffold the three-service structure quickly. Then I switched to GitHub Copilot for the final submission to clean it up and do the precision work.

I've used Claude Code for documentation-heavy tasks — things like generating an architecture.md or writing a DECISIONS.md that walks through why I made each design call. It's very good at reasoning about design when you give it the actual code as context.

Windsurf I've played with but it's less integrated into my daily flow — it's strongest for greenfield projects where you want the IDE to own more of the scaffolding decisions.

---

3. The Take-Home Project (Consumer Trend Intelligence Agent)

Q: Walk me through what you built for the take-home project.

I built a **Consumer Trend Intelligence Agent** — a tool where a marketer types a question in plain English like "how is the energy drink market trending in Texas?" and gets back a structured intelligence brief with source citations.

The system has three layers:

1. **MCP Tool Server** (FastAPI, port 8001) — this is a stateless microservice that owns all external I/O: YouTube Data API, NewsAPI, and US Census ACS. The agent backend never calls those APIs directly — it always goes through this server. API keys are isolated here.
2. **Agent Backend** (FastAPI, port 8000) — this is where the LangChain tool-calling agent lives. When a question comes in, it first classifies the intent (is this a sentiment question? competitive? market sizing? campaign ideas?), then builds a targeted prompt and a filtered tool list, then runs the agent loop, and finally returns a structured response with the brief, source citations, and the tool trace.
3. **Frontend** — a simple HTML/JS SPA served statically by the agent backend. Streaming responses, source cards with clickable links, a mode badge showing which intent was detected.

The design choice I'm most proud of is **evidence gating** — the agent is instructed to only write a section of the brief if it actually retrieved data to back it up. So if it didn't get brand-level signals, it skips the Market Stats section and documents the omission in the Caveats section instead of hallucinating. That was a deliberate call to build trust into the output.

Q: Why did you build the MCP server as a separate service instead of just calling the APIs inline?

Two main reasons.

First, **security and isolation**. The external API keys — YouTube, NewsAPI, Census — live only in the MCP server's environment. The agent backend never sees them. If someone exploits the agent backend, they can't get those credentials. The hard process boundary enforces that contract.

Second, **testability**. In my tests, I point `MCP_SERVER_URL` at a local fixture server that returns fake data. The agent backend's full logic — intent classification, tool selection, prompt building, response formatting — runs against that fixture without touching any real API. If I had inlined the API calls into the tool functions, I couldn't test the orchestration layer without mocking at a much lower level.

The cost is a local HTTP round-trip on every tool call, which is about 5ms on localhost. Totally acceptable for a demo or low-volume tool. In production at higher throughput, I'd add caching inside the MCP server first before questioning the architecture.

Q: Walk me through how the intent classification works.

When a question comes in, I run it through a regex pattern matcher before touching the LLM. I have a priority-ordered pattern list for each intent — for example, competitive intent looks for patterns like `\bvs\b`, `compare`, `competitor`, `market share`, `head-to-head`. The first pattern that matches wins.

If nothing matches, I fall back to a single lightweight LLM call — just a short classification prompt, not the full agent loop. That keeps the common case fast (regex is microseconds) and the edge cases still handled.

The reason I do this pre-routing instead of just exposing all eight tools to the agent every time is tool budget. I cap the agent at five tool calls. If it has all eight tools available and a sentiment question comes in, it might waste two calls on `get_market_demographics` which it doesn't need. Pre-routing narrows both the tool list and the output sections to exactly what the intent requires, so the agent's decisions are more predictable and the output is tighter.

Q: What would you do differently if you had more time or were building this for production?

A few things:

Caching. Right now every question hits the live APIs. Google Trends signals for a given query barely change in 15 minutes, so caching MCP tool responses in Redis with a short TTL would cut latency and API costs significantly.

LangGraph instead of AgentExecutor. The current loop is linear — call tools, synthesize, return. But a real production version would benefit from branching: if the market stats tool comes back empty, route to a fallback strategy instead of just noting the caveat. LangGraph's explicit graph nodes would make that control flow transparent and debuggable. I called this out in my DECISIONS.md as the clear migration path.

Rate limiting and auth on the MCP server. Right now the MCP server accepts any request on localhost. If you expose it externally, you'd want API key validation and per-client rate limiting on those endpoints.

Streaming the tool trace live. Currently the response streams the final text. A better UX would stream each tool call result as it comes in so the user can see the agent working in real-time.

Q: How did you use the agentic IDE specifically to build this project?

I'll be honest about the workflow: I used GitHub Copilot as a tight feedback loop throughout. The pattern was:

1. **Design first** — I sketched the three-service architecture and the data flow on paper before opening the IDE. The DECISIONS.md was written early, not after. This gave Copilot enough context to work with.
2. **Scaffold one, replicate many** — Once the first MCP tool endpoint

(`search_youtube_discussions`) was solid, I used Copilot to replicate the pattern across the other seven tools. That took about two hours for all eight.

3. **Prompt engineering iteration** — For the system prompt in `prompts.py`, I iterated heavily with Copilot Chat. I'd show it a sample LLM output, describe what was wrong ("it's putting URLs in a list instead of inline"), and ask it to revise the guidance block. That back-and-forth tightened the prompt significantly.
4. **Test generation** — I used Copilot to generate the initial test stubs for `test_agent_fallback.py` and `test_mcp_models.py`, then filled in the assertions manually.

---

4. LLM & Agent Architecture

Q: What is tool calling / function calling and how does it work?

It's the mechanism that lets an LLM decide to call external functions as part of answering a question rather than just generating text from its training data.

You define a set of tools — each has a name, a description, and a structured input schema. That schema goes into the LLM's API call. When the model's response includes a tool call instead of a text answer, your code intercepts it, runs the actual function, passes the result back to the model as a new message, and the model continues. This loops until the model produces a final text answer with no more tool calls.

In my project, the tools are things like `search_youtube_discussions` or `get_trend_timeline`. The agent decides which ones to call and in what order based on the question and the system prompt. I used LangChain's `create_tool_calling_agent` which handles the dispatch loop and the intermediate step collection automatically.

Q: What is MCP (Model Context Protocol) and why did you use it?

MCP is a protocol for structuring how an LLM agent calls external tools or services. Instead of just writing Python functions that the agent calls in-process, you define tool endpoints as a separate service with a consistent interface — named tools, structured inputs, structured outputs over HTTP.

The benefit is the same reason you'd build a REST API instead of a library: the contract is explicit and the implementation is swappable. Any client that speaks MCP can use those tools, not just my specific agent. And the tools themselves can be updated, scaled, or mocked independently.

In my project I implemented the MCP server as a FastAPI service with one endpoint per tool. The agent backend calls it over HTTP JSON-RPC. It's not the official Anthropic MCP SDK in this case — it follows the same architectural pattern — but the concept is identical. I've used the actual LangChain MCP adapter at Samay Systems on real production infrastructure.

Q: When would you use RAG vs. tool calling vs. fine-tuning?

These solve different problems:

Tool calling is the right choice when the information you need is live — current news, real-time API data, user-specific records. The LLM can't have this in its training data because it didn't exist. That's what I built in the take-home: live YouTube, NewsAPI, Census data retrieved at query time.

RAG is right when you have a large, relatively static corpus that you want the LLM to reason over — internal docs, a knowledge base, contracts. You chunk it, embed it, store it, and retrieve the relevant chunks at query time. At Samay Systems, I built two layers of this. The first was a GraphRAG chatbot that sat on top of a Neo4j graph: a scheduled pipeline ingested identity events from GitHub, JIRA, and EntraID into graph nodes and relationships, then a Flask service running on AWS AppRunner translated natural language questions into Cypher queries using LangChain — so a compliance analyst could ask "which users have access to this repo and what controls govern that?" and get a graph-traversal answer. The second layer was document embeddings: compliance policy documents were stored in S3, their content was chunked and embedded, and the embeddings were used to pull relevant policy context into the control-testing prompts at analysis time.

Fine-tuning is the right choice when you need the model to change *how it talks* — a specific tone, a domain-specific format, consistent structured output for a narrow task — not *what it knows*. It's expensive and slow to iterate, so I only reach for it when prompt engineering has genuinely maxed out.

In practice, most internal tools are tool calling + RAG. Fine-tuning is rare.

Q: How do you prevent an LLM from hallucinating in a production system?

I don't try to stop hallucinations at the model level — that's a losing battle. I change the system so hallucinations can't get into the output unchallenged.

The main mechanisms I use:

Evidence gating in the prompt — the system prompt says "write this section only if you have retrieved data to support it." If the model has nothing to cite, it skips the section and documents the omission. This is exactly what I built into the take-home project. A shorter honest report beats a longer fabricated one.

Source extraction and display — every response shows the actual URLs and titles of the sources the agent used. The user can click through. If the LLM is making something up, there's no source card for it — that's a visible signal.

Structured output schemas — using Pydantic models as output validators means the model can't return free-form text that sneaks in unsupported claims. It has to put confidence caveats in the designated Caveats section.

Low temperature, not always zero — I run interactive agents at temperature 0 because I want the same question to produce consistent output. For batch LLM analysis jobs — like the automated control testing at Samay, where Bedrock was analyzing CloudTrail logs and Security Hub findings against a compliance control — I used temperature 0.1. Just enough to avoid deterministic

repetition across a batch of similar inputs, but still firmly in the factual register. The model choice matters too: I used Llama 3.3 70B Instruct on Bedrock for that, specifically because it's strong at structured reasoning and the 70B scale handles multi-page data summaries reliably.

---

5. Systems Thinking & Execution

Q: How do you turn an ambiguous business problem into a working system?

My process: **decompose, then constrain, then build in the smallest slice that proves the thing works.**

When someone says "we need to automate how we track vendor compliance," I don't start coding. I ask: what is the specific input? What is the specific output? Who consumes it? What breaks if it's wrong?

Then I draw the data flow — where does data come from, where does it go, what transforms it. That usually reveals two or three key decision points: do we call the vendor API or ingest their exports? Do we store the result in the existing DB or a new one? I write those decisions down with the rationale before touching code.

Then I build the narrowest possible vertical slice — one input, one transformation, one output — and get it in front of a real user. The feedback from that first working thing is worth more than all the speculation that came before it.

At Samay Systems I did this for the automated compliance testing pipeline. The full vision was: given any compliance control ID, automatically collect evidence from six AWS services (CloudTrail, Config, Inspector, Security Hub, VPC Flow Logs, CloudWatch), pull integration data from GitHub/JIRA/EntraID, run LLM analysis against the combined data, and generate a scored compliance report per tenant. That's a lot of moving parts.

Instead of building all of it at once, I started with one control (CC7.1 — system monitoring) and hardcoded most of the parameters. Got that working end-to-end: one Lambda, one LLM call to Bedrock, one S3 output. Showed it to the team. They changed two of the evidence sources based on what they saw. Then I generalized: added AWS Step Functions to orchestrate the per-tenant Lambda invocations in parallel, made the control ID a dynamic input, added multi-tenant resolution from RDS. The final system could test any control across all tenants with one trigger. But it started as a hardcoded script for one control and one tenant.

Q: Talk about a time you had to move fast without full requirements.

The take-home project is actually a good example. The brief said "build an agentic web app with an MCP server and LangChain tool calling" — that's the skeleton, not the full spec. I had to decide: what public API is interesting and defensible? What does the user actually want out of this?

I picked consumer trend intelligence because it was domain-specific enough to demonstrate real

tool-calling logic — the LLM needs to make multi-step decisions, not just one fetch. And it solved a real problem that marketers have.

I built the rough prototype first (the `AI Builder Challenge` folder) in about a day to test that the core loop worked end-to-end. Once I knew it worked, I rebuilt the submission cleanly with proper architecture, ADRs, and tests. The prototype was throw-away scaffolding; the submission was the real thing.

The discipline there is: don't polish the prototype. Use it to answer the question "does this actually work?", then throw it away and build properly.

Q: Give me an example of moving quickly from a vague mandate to something working — in a real job, not the take-home.

At Epik, the brief was literally "automate SOC workflows." That's not a spec, that's a direction. The mandate came from leadership; I had to figure out what it actually meant.

Day one, I sat with the security analysts for two hours. Not to gather requirements — just to watch them work. The biggest friction point was clear within the first 30 minutes: every morning they were manually reading through an overnight queue of hundreds of alerts, most of which were the same class of low-value noise. The decision they had to make on each one — act, ticket, or close — was repetitive and slow.

I sketched a three-bucket classifier on a whiteboard that afternoon with the team lead: low confidence gets auto-closed with a logged reason, medium creates a JIRA ticket with context, high pages an on-call analyst. Everyone agreed that was the right shape. I had a working proof-of-concept classifying live events by end of the week — Azure OpenAI for the classification prompt, n8n for the routing. The output wasn't pretty, but it was doing the right thing on real data.

Then we spent two weeks running it alongside the humans in shadow mode — it would make a recommendation, the analyst would make the actual call, we'd compare. That let us calibrate the thresholds based on real disagreements. After two weeks the false-negative rate was low enough to trust it. Full handoff happened in under a month from the original mandate.

The lesson for me was: **the fastest path to a working system is watching the person who currently does the job by hand.** Requirements documents tell you what people think they want. Watching them work tells you what actually hurts.

Q: The JD mentions workflow orchestration and data pipelines. What's your experience there?

This is actually a big chunk of what I've built.

At Samay Systems, I designed the **Profile-360 pipeline** as an AWS Step Functions state machine. The job was: for each tenant, pull identity and activity data from GitHub, JIRA, and EntraID; format it; and ingest it into that tenant's Neo4j graph. Each of those steps was a separate Lambda function. Step Functions handled the sequencing, retries, and error states. A scheduler Lambda triggered the whole machine on a cron. The reason to use Step Functions instead of just chaining

Lambda calls was visibility — you can see exactly which step failed, what the input/output was, and replay from that point. That's invaluable when a third-party integration flakes out at 2am.

At Epik, I used **n8n** deployed on ECS Fargate for the incident triage workflow. n8n is the right tool when the orchestration logic changes frequently and you want non-engineers to be able to adjust routing rules without a code deploy. The actual classification logic was still an LLM call, but the routing — "if severity is high, page this Slack channel; if it's medium, create a JIRA ticket" — lived in n8n where the ops team could tweak it directly.

The practical distinction I use: Step Functions for pipelines where reliability and auditability matter most, n8n for workflows where the logic needs to change fast and the consumers aren't engineers.

Q: How do you evaluate whether a problem actually needs AI or if traditional software would do the job better?

AI earns its place when the input is unstructured or highly variable and the rules to process it would be too brittle to maintain as code.

If I can write a `if/else` tree that handles 95% of cases reliably, I do that. It's faster, cheaper, more debuggable, and doesn't hallucinate.

Where AI wins: extracting structure from free-form text (like a user question → structured intent), reasoning over variable-length documents, generating narrative output that sounds human. These are things where rule-based systems break the moment you hit an edge case.

In my intent classification for the take-home, the first pass is regex — fast, deterministic, testable. The LLM only fires on the cases regex can't handle. That's the right split: use the simplest tool that works for the common path, escalate to AI for the long tail.

A bad use of AI: using an LLM to validate that an email address has an @ sign. A good use: using an LLM to triage which of 1,000 daily security events are worth a human looking at, because the rules for that are too complex and too variable to maintain as code. That's exactly what I built at Epik.

---

6. Past Projects & Real-World Experience

Q: Tell me about a real AI system you built that actually got used. What was the impact?

At Epik Solutions, I built an AI-driven incident triage pipeline for a security operations team. The problem was straightforward but painful: the team was getting 1,000+ security events a day, and analysts were manually reading through them to decide which ones needed a human response.

I built a Python pipeline on top of Azure OpenAI that read each event, ran a structured classification prompt to assign severity and category, and then routed it — auto-close for known low-value events, create a ticket for medium, page an analyst for high. The pipeline ran on n8n for orchestration, which made it easy to adjust the routing logic without touching code.

The result was about a 30% reduction in analyst workload. More importantly, the average time-to-response on real incidents went down by 15 minutes because analysts weren't spending the first part of their shift triaging noise.

The hardest part wasn't the AI — it was calibrating the thresholds. Too aggressive on auto-close and you miss real issues; too conservative and you're back to the same noise problem. We did a two-week shadow period where the pipeline ran alongside the humans before we trusted it to act autonomously.

Q: Walk me through a second AI project from a past role — problem, approach, results.

At Samay Systems, compliance auditors were spending a full day or more on each compliance control during audit season — manually pulling logs, screenshots, access records, formatting evidence packets. The problem wasn't that they were slow; the work was just inherently tedious and repetitive.

The ask was to automate it. But "automate compliance evidence collection" is still vague — the interesting question is *what evidence*, from *which sources*, in *what format*.

I spent time with the auditors to understand what they actually looked at for a specific control. For CC7.1 (system monitoring), the evidence they cared about was: were config changes tracked? Were vulnerabilities detected and actioned? Were access changes logged? That mapped directly to specific AWS services — CloudTrail for config change events, AWS Inspector and Security Hub for vulnerability findings, EntraID events from our identity pipeline for access changes. Three more sources came from GitHub, JIRA, and our internal document embeddings for policy context.

I built a Lambda function that collected from all six sources, summarized the data, and passed it to AWS Bedrock (Llama 3.3 70B) with the control's requirements as context. The model returned a structured compliance score, key findings, and gaps. Results were written to an S3 bucket. AWS Step Functions orchestrated the whole thing per tenant — so you could trigger it for all clients with one invocation and it would fan out.

The outcome: what previously took an auditor most of a day now ran in minutes on a schedule. Clients could see their compliance posture against a control *before* audit season, not just during it. That shift — from reactive evidence gathering to continuous monitoring — was the actual business value.

Q: Tell me about a time you built internal tooling for non-technical users. How did you design for that?

At Epik I built React dashboards for the C-suite that aggregated telemetry from 15+ security systems. The users were executives — they cared about "are we getting worse or better?" and "what do I need to act on today?", not about individual alert counts.

The first version I built was too granular — I gave them all the numbers I had. They used it for a week and stopped. When I asked why, they said "we don't know what to do with this."

So I rebuilt it around three things: a traffic-light status for each system, a "needs attention" list with plain-English explanations, and a week-over-week trend line. That's it. Everything else was a click-through.

That version got used. The lesson: internal tooling for non-technical users needs to answer the question "what should I do right now?" before it answers "here's the data."

Q: You have experience with both AWS and Azure. How do you decide which cloud services to use for an AI workload?

Practically, the first factor is where the organization already is — I'm not going to recommend a five-service AWS Bedrock setup to a company that's all-in on Azure, because the operational overhead of multi-cloud for a single workload is rarely worth it.

Once you're within an ecosystem: for LLM inference specifically, I'd reach for a managed API endpoint (Azure OpenAI, AWS Bedrock, or direct Groq/Anthropic) rather than self-hosting a model unless I have a very specific reason — cost at extreme scale, data residency, or a custom fine-tune that the managed services can't host.

For everything around the LLM — the orchestration layer, the tool servers, the databases — I try to use serverless where the workload is bursty (Lambda, Azure Functions) and containers where the workload is sustained or needs warm-start latency guarantees (ECS, AKS).

Q: How do you approach database and data modeling decisions? What's your experience with SQL and NoSQL?

I've worked with relational, graph, and document stores in production, and the modeling decision has mattered each time.

Relational — PostgreSQL at Samay: The multi-tenant architecture used RDS PostgreSQL as the authoritative source for tenant metadata: which tenant ID maps to which Neo4j instance, which S3 bucket, which service configuration. The schema was intentionally simple — a `tenants` table with foreign-keyed configuration rows — but the discipline was strict: all tenant resolution logic lived here and nowhere else. Every Lambda function that needed to act on a specific client's data queried RDS first to get the mapping, then derived everything downstream from that. No hardcoded tenant identifiers, no duplicated resolution in application code. When you're serving 15+ enterprise clients, a tenant routing inconsistency is a production incident that crosses data boundaries.

Graph — Neo4j at Samay: For the identity and access intelligence layer, relational was the wrong shape. The queries that mattered to auditors — "which users have access to this repo and what changed in the last 30 days?" — are multi-hop traversals across entities. I modeled identity events as nodes with timestamp properties and typed relationships: `HAS_ACCESS_TO`, `CHANGED_ROLE`, `BELONGS_TO_ORG`. That structure made Cypher queries for compliance checks clean and readable, and it's why the NL → Cypher translation worked well — graph traversal patterns map naturally to how auditors think about access chains.

The decision rule: Relational for structured records with consistent schemas and transactional

consistency requirements. Graph when the most important queries are about multi-hop relationships between entities. Document DB when records are schema-variable or hierarchically nested. I try not to let familiarity with one tool drive the choice — the wrong model makes every query harder.

---

7. Deployment, DevOps & Cloud

Q: How would you deploy the take-home project in production?

I gave a verbal overview of this in the video, but the concrete plan:

Containerize each service separately. The MCP server and agent backend each get a `Dockerfile`. They're independent services that should be deployable and scalable independently — the MCP server might need more horizontal scale if tool-call volume is high.

Docker Compose for local dev and staging. One `docker-compose.yml` that starts both services with environment variables from a `.env` file. This is also what a new developer runs on day one — one command, everything up.

For production: The MCP server is a good candidate for a serverless container (Azure Container Apps or AWS Fargate) because its calls are stateless and bursty. The agent backend could be the same, though you'd want to watch cold-start latency if agents take 2–3 seconds to warm up.

Secrets management: API keys go into a secrets manager (Azure Key Vault or AWS Secrets Manager), not `.env` files. The container pulls secrets at startup, not at build time.

Infrastructure as code: I wouldn't manage any of this by clicking through the AWS console. At Samay Systems I wrote all the infrastructure in AWS CDK (TypeScript) — one `cdk deploy --all` stood up the VPC, RDS, all the Lambda stacks, the AppRunner service, and the ECS Fargate services for a given environment. The same CDK code deployed dev, staging, and prod by passing an `--context env=` flag. That pattern is what I'd bring to this project: Docker images define the runtime, CDK or Terraform defines the infrastructure, everything is in source control and reproducible.

CI/CD: GitHub Actions pipeline that runs the test suite on every PR, builds the Docker images on merge to main, and pushes to a container registry. The environment promotion (staging → prod) is a manual approval gate.

Q: How do you handle API keys and secrets in a multi-service system?

Hard rule: secrets never live in code, never in environment files committed to source control, and never in container images.

In the take-home project, I use a `.env` file that's gitignored — that's fine for local dev. The `.env.example` that's committed shows the shape without real values.

For production systems, I've used Azure Key Vault at Epik and AWS Secrets Manager at Samay Systems. Both have the same pattern: the application has an identity (managed identity or IAM role)

that grants it read access to specific secrets. The app fetches the secret at startup and caches it in memory. Rotation is handled at the secrets manager level — you don't redeploy the app, you rotate the key and the app picks it up on next startup or cache expiry.

The MCP server pattern in my take-home is actually the right first step: external API keys are isolated to one service. If the agent backend is compromised, the attacker can't directly read the YouTube or NewsAPI keys — they'd have to also compromise the MCP server's environment.

At Samay, the multi-tenant secret pattern in AWS Secrets Manager was more granular than just one key per service. Every tenant had its own Neo4j credentials stored under a path like `neo4j/tenant-{id}`. The AppRunner service and Lambda functions resolved the tenant ID from RDS first, then fetched only that tenant's credentials from Secrets Manager. An IAM policy scoped to `arn:aws:secretsmanager:...:secret:neo4j/*` meant the service could fetch any tenant's secret, but it could only ever query the Neo4j instance it resolved to — the credential itself enforced the isolation boundary. That kind of layered isolation — tenant resolution, scoped IAM, per-tenant secrets — is the right model for any multi-tenant system handling sensitive data.

---

8. Trade-offs & Decision Making

Q: Talk me through a technical decision you made that had real trade-offs. What did you give up?

The decision to do intent classification with regex + LLM fallback instead of just routing everything through the full agent was a meaningful trade-off.

What I gained: Speed and predictability. The regex path takes microseconds. The tool list is narrow. The agent's decisions are more constrained and easier to reason about. In evaluation, the output was more focused when the intent was correctly classified.

What I gave up: Flexibility. A question like "how is Monster doing in Texas and what should a campaign look like?" sits at the intersection of competitive and campaign intents. My pattern priority order decides which one wins, and that decision is invisible to the user. They get a valid response but scoped to one intent. A smarter multi-intent decomposition would handle this better.

I called this out in the DECISIONS.md. The mitigation is that `full_brief` is the safe fallback for anything ambiguous — so the worst case is a broader report, not a missing one. In production I'd add an intent confidence score and route low-confidence classifications to `full_brief` automatically.

Q: What's an example of "progress over perfection" in your work?

At Samay Systems, when we were building the compliance automation, we had a choice between two architectures for the evidence collection pipeline: a simple sequential Python script per control type, or a full distributed workflow with DAG-based orchestration.

I built the sequential scripts first. They were ugly — no retry logic, no parallelism, not production-ready. But they ran in a week and we got real evidence collection working for three control types. We demoed that to the client.

The client came back with: "we actually don't need these three control types. These other two are what we care about." If I had spent three weeks building the distributed orchestration system around the wrong control types, that feedback would have been devastating.

We shipped the proper architecture in the second month, informed by what we learned. The ugly scripts were the fastest way to find out what we were actually building.

The same principle applies to data and ML work: early experiments belong in a notebook. When I was validating the LLM analysis approach for the control testing system, I ran the initial iterations in Jupyter — paste a data sample, adjust the prompt, inspect the output inline, iterate. Fast loop, no deployment overhead. Once the approach proved out, I moved it into the Lambda service: Pydantic output schemas, error handling, unit tests. Notebooks are for *discovering*; deployed functions are for *shipping*. The discipline is knowing when to cross that line.

Q: Tell me about a time you inherited or dealt with technical debt. How did you approach modernizing it?

At Samay, when I joined as the founding engineer, the infrastructure existed but was partly hand-provisioned — click-ops through the AWS console, no IaC. There was no reliable way to spin up a clean environment. Dev had drifted from staging, staging had drifted from prod. New developers were spending days asking "where is this configured?" because the answer was "someone's browser history."

I migrated everything to AWS CDK (TypeScript) incrementally — never all at once, which would have introduced too much risk. The pattern was: pick one resource, write the CDK construct to match its current state, import it into CDK management, deploy to a clean test environment to validate the construct, then cut over. Stack by stack. VPC first, then RDS, then each Lambda stack, then AppRunner, then ECS Fargate. By the time I finished, 15+ stacks were CDK-managed and one `cdk deploy --all` stood up a complete environment from scratch. Environment promotion became a flag: `--context env=staging`, `--context env=prod`.

The second example is Epik: security health data that was being tracked manually in spreadsheets — incident counts by source system, SLA compliance rates, system health — became automated ingestion pipelines feeding React dashboards. That eliminated 20+ hours of weekly manual reporting and made the data actually current instead of last Thursday.

The common pattern in both cases: I didn't try to rewrite everything at once. I looked for the chokepoints that were causing repeated manual work or reliability problems, fixed those first, and made the system more self-explanatory at each step. Technical debt pays compound interest the longer you ignore it — but the fastest path to a better system is picking the most painful thing first, not redesigning everything.

---

9. Communication with Non-Technical Stakeholders

Q: How do you explain what an LLM agent does to someone who doesn't code?

I usually use a staffing analogy.

"Think of the agent like a very fast researcher you can ask questions. When you ask it something, it doesn't just answer from memory — it knows how to use tools, like searching YouTube for what people are talking about, or checking the news, or looking up market data. It decides which tools to use, runs them, reads the results, and then writes you a summary. You get an answer backed by things it actually looked up just now, not something it's guessing at."

That usually lands. The important thing to get across is: the LLM doesn't know your specific question's answer ahead of time; it's reasoning about *how* to find it and then synthesizing what it found.

Q: How do you handle it when a non-technical stakeholder wants a feature that you know is technically problematic?

I don't say "that's not possible" — that's rarely true and it's not helpful. I say "here's what that would take, and here's what we'd have to give up."

At Epik, a stakeholder wanted the incident triage pipeline to send Slack alerts with full event details — every field, all the context. The problem was that some of those fields contained PII from employee accounts.

I walked them through it: "We can send Slack alerts with the fields that are safe. The full detail view would need to be a link to the internal ticket system, which is already access-controlled. That also means the alert itself is shorter and more actionable." They actually preferred that outcome once they understood the constraint.

The goal is to give them the business outcome they actually want, not the specific feature they asked for.

---

10. Behavioral / Situational

Q: Tell me about a time something you built broke in production. How did you handle it?

At Samay Systems, we had a distributed identity event ingestion pipeline that was silently dropping events under high load. No errors, no alerts — the pipeline looked healthy from the outside, but the event counts were off. We found it because an auditor noticed that a user's access history had gaps.

I ran it down by adding counters at every stage of the pipeline — events received, events processed,

events written — and comparing the deltas. The failure point was a Spark job that was hitting memory limits and silently discarding the overflow batch instead of failing with an error. It had been configured with error handling that was too defensive.

Fix was two parts: reduce batch size so memory pressure was manageable, and change the error handling so memory failures raised visibly rather than being swallowed. We also added an end-of-pipeline count reconciliation job that ran after each batch and paged if the numbers didn't match.

The lesson: **silent failures are worse than loud ones.** A system that fails invisibly erodes trust slowly. I'm now very deliberate about making failure states noisy.

Q: How do you handle learning something new on the fly in a project?

I lean into it. My approach is: find the smallest running example of the thing I need to understand, run it, break it deliberately, fix it, then adapt it to my use case.

When I needed to implement GraphRAG at Samay — I hadn't used Neo4j for graph traversal in a RAG context before — I spent four hours on the docs and a toy example before touching the real codebase. That toy example was throw-away, but it gave me enough to move confidently.

I also use AI tools heavily for this. If I'm in an unfamiliar library, I'll paste a code snippet and ask Copilot "explain what this is doing and what the common pitfalls are." That compresses the ramp-up time significantly.

The honest constraint: there are things that take real experience to get right — distributed systems failure modes, auth architecture, performance tuning. For those I'm upfront about what I know well and where I'd want to talk to someone with more depth.

Q: You've worked as a founding engineer at a startup. How did you manage building multiple things at once without dropping anything?

At Samay, at any given point I was running the Profile-360 pipeline, the automated control testing Lambda, 8+ frontend feature branches, and CDK infrastructure changes simultaneously. Those are four completely different contexts — distributed systems work, LLM prompt engineering, React UI, TypeScript infrastructure code.

A few things that kept it manageable:

One hot context per day. I tried hard not to switch between deep contexts within the same working block. If I was in Lambda/Python mode, I'd stay there until that work was at a natural stopping point. Context-switching between infrastructure CDK code and React UI in the same afternoon is the fastest way to make mistakes in both.

Batch similar infrastructure work. CDK stack deployments are expensive in terms of time — a `cdk deploy --all` takes 10-15 minutes. I'd accumulate several smaller infra changes and do one combined deploy rather than deploying after each individual tweak.

Use AI tools to restore context. Augment Code's cross-repo awareness was genuinely useful here — when I came back to a Lambda branch after two days on frontend work, I'd ask it to summarize

the current state of that branch and what was in flight. That cut the "okay where was I" ramp-up from 20 minutes to about 5.

Explicit end-of-day notes to myself. Each open branch got a one-line status note: "blocked on S3 permissions fix," "ready for review," "needs prompt revision." Small thing, but it meant I could triage in the morning without re-reading all the code.

The honest answer is that startups require accepting some switching overhead — you can minimize it but not eliminate it.

Q: Tell me about a time you had to work closely with a non-engineering team to figure out what you were actually building.

At Samay, I couldn't have built the automated compliance testing system just by reading a spec — the domain was too specialized. Compliance controls read like legal documents. "The organization monitors for unauthorized configuration changes" doesn't tell you what data you need or what "passing" looks like.

So before writing any code, I sat with the compliance auditors for a few sessions. Not a requirements meeting — more like shadowing. I asked them to walk me through an actual audit of one control, step by step, as if I was a new auditor learning the job. What do you open first? What are you looking for? What would make you flag this as non-compliant?

That conversation was where the 6-source data model came from. They didn't say "I need CloudTrail data" — they said "we need to see that config changes were tracked and reviewed." I translated that into: CloudTrail for the change log, AWS Config for compliance rule states. They said "we check vulnerability management" — that became Inspector v2 findings and Security Hub. They said "we look at who had access to critical systems" — that became EntraID events from the Profile-360 pipeline.

If I had just guessed at the data sources from the control text, I'd have built something technically correct that didn't match what auditors actually needed. The human conversation was the spec. After that, building it was the easy part.

Q: Tell me about a time requirements shifted significantly mid-build. How did you handle it?

At Samay, I was mid-build on the automated control testing system — had CC7.1 working, was generalizing the Lambda to accept any control — when the team lead came back from a client call with new priorities. The client needed a different set of controls tested first, ones that required a slightly different evidence profile.

My first reaction was frustration. But when I actually looked at the impact, it was smaller than I expected. Because I'd built the data collectors as separate modules — `aws_collector.py`, `integration_collector.py` — and the LLM analysis was a function that took a control ID and a data payload, most of the wiring was already generic. Swapping which control was being tested meant updating the prompt context and tweaking which collectors were called. It wasn't a rewrite.

I gave the team lead a realistic assessment: "I can have the first of those controls running by end of week, the second the week after." Not "this is going to blow up the timeline," not "sure, no problem" without thinking it through. An honest estimate with the reasoning behind it.

The broader lesson: **requirements will always change — the question is whether your architecture can absorb it.** When I start a new system, one of the first things I think about is: which parts of this are most likely to change, and am I coupling them to the parts that aren't? That habit has paid off more than once.

Q: How do you prioritize when you have multiple tasks and unclear urgency?

Impact times reversibility. I ask: which of these has the highest business impact? And which is hardest to undo if I get it wrong?

High-impact and hard-to-undo goes first, with the most care. High-impact and easy-to-undo is fast because I can iterate. Low-impact stuff gets batched or dropped if the list is long.

In practice I use a simple daily list — three things I need to ship today, two things I want to make progress on, everything else is background. I learned the hard way that a backlog you actually use looks like ten items, not fifty.

Q: What do you genuinely want out of this role?

I want to build things that actually make people's jobs easier — not demos, not proof-of-concepts that never ship. The AI Builder role at InMarket is specifically about building internal tools that teams use. That feedback loop — you build it, a real person uses it, you see whether it helped — is the most honest test of whether the engineering actually matters.

I also want to go deeper on the agentic side. Building the take-home project showed me that the interesting design problems are at the agent level: how do you structure tool use, how do you constrain the model's decision space, how do you make the output trustworthy. I want to work on that in a real business context, not just toy projects.

Quick Reference — Key Numbers to Remember

Project / Role	**Key Metric**
Epik — triage pipeline	1,000+ daily events, 30% analyst workload reduction, 15 min MTTR improvement
Epik — dashboards	15+ source systems, eliminated 20+ hrs/week manual reporting
Epik — automation	500+ users, 40% reduction in operational cycle times
Samay — compliance SaaS	15+ enterprise clients, 10,000+ users per org, 70% auditor effort reduction

Project / Role	Key Metric
Samay — identity pipeline	5M+ daily events, GraphRAG chatbot (NL→Cypher via AppRunner), 30% reduction in unauthorized access footprints
Samay — automated control testing	Lambda + Step Functions + Bedrock (Llama 3.3 70B), data from 6 AWS services + 3 integrations, per-tenant compliance scoring
Samay — AWS infrastructure	CDK TypeScript, 15+ stacks (VPC, RDS, Lambda, ECS Fargate, AppRunner), dev/stg/prod from one codebase
Monarch Tractor — CV/ML	15% object detection accuracy improvement, 20ms latency cut, 500+ deployed units
Take-home project	3-service architecture, 8 MCP tools, 5 intent classes, Groq + Gemini fallback

---

Things NOT to Say

- Don't over-explain LangChain internals unless they drill in — lead with what it does, not how it works
- Don't apologize for the "entry-level" positioning — you cleared the take-home, you're in the tech round
- Don't say "I don't know" cold — say "I haven't hit that specific scenario but here's how I'd approach it"
- Don't conflate MCP the architecture pattern with the Anthropic MCP SDK — the take-home uses the pattern, not the SDK
- Don't pad answers — if you've answered the question, stop. They'll drill in if they want more.

Last updated: May 2026 — InMarket AI Builder tech interview prep